Natural Language and
Information Retrieval
Degree in Data Science
Academic year 2024-2025

Lab 2: Sexism Identification in Twitter

Session 3. Transformers: Fine-tuning for multi-label classification.

Goal

The goal of this lab session is to help students understand and gain practice in the use of deep learning-based language models, in particular transformer-based models (BERT, RoBERTa) for English and Spanish. The second objective is the application of these models to text classification tasks, particularly for the third subtask included in the shared task *"EXIST2025: sEXism Identification In Social neTworks"*. This subtask focuses on multi-label classification by categorizing diverse types of sexism within sexist tweets, enabling a nuanced analysis of sexist language and its manifestations.

Bibliography

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT press.
2. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. Advances in neural information processing systems, 30.
3. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. NAACL-HLT 2019, pages 4171–4186, ACL.

4. Cañete, J., Chaperon, G., Fuentes, R., Ho, J. H., Kang, H., & Pérez, J. (2020). Spanish pre-trained bert model and evaluation data. In Proceeding of the Practical ML for Developing Countries Workshop @ ICLR 2020, pages 1-9, Addis Ababa, Ethiopia.
5. De la Rosa, J., Ponferrada, E., Romero, M., Villegas, P., González de Prado Salas, P., & Grandury, M. (2022). BERTIN: Efficient Pre-Training of a Spanish Language Model using Perplexity Sampling. Procesamiento Del Lenguaje Natural, 68, 13-23
6. Cañete, J., Donoso, S., Bravo-Marquez, F., Carvallo, A., & Araujo, V. (2022). Albeto and distilbeto: Lightweight spanish language models. arXiv preprint arXiv:2204.09145.
7. Gutiérrez-Fandiño, A., Armengol-Estapé, J., Pàmies, M., Llop-Palao, J., Silveira-Ocampo, J., Pio Carrino, C., Armentano-Oller, C., Rodriguez-Penagos, C., Gonzalez-Agirre, A., & Villegas, M. (2022). MarIA: Spanish Language Models. Procesamiento Del Lenguaje Natural, 68, 39-60.
8. https://huggingface.co/docs/transformers/index
9. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Weizhu Chen: LoRA: Low-Rank Adaptation of Large Language Models**.** CoRR abs/2106.09685 (2021)
10. Mangrulkar S., Gugger S., Debut L., and Belkada Y., and Paul S., and Bossa B. (2022). PEFT: State-of-the-art Parameter Efficient Fine-Tuning methods. https://github.com/huggingface/peft.

11. Han, Z., Gao, C., Liu, J., & Zhang, S. Q. (2024). Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*.
12. Ayupov, S., & Chirkova, N. (2022). Parameter-efficient fine tuning of transformers for source code. Workshop on Efficient Natural Language Processing at NeurIPS 2022
13. Amigo E., and Delgado A. 2022. Evaluating Extreme Hierarchical Multi-label Classification. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 5809–5819, Dublin, Ireland. Association for Computational Linguistics.

MULTI-LABEL CLASSIFICATION

Multi-label classification is an advanced machine learning problem in which a single input instance can be simultaneously associated with multiple, non-mutually exclusive labels. Unlike traditional classification approaches that require an instance to belong to a single class, multi-label classification allows for a more nuanced and complex labeled data representation. In this approach, each input can be tagged with zero, one, or multiple labels, reflecting the inherent complexity of real-world classification problems. Typical applications include document categorization (where a text can belong to multiple topics), image annotation (an image containing multiple objects), medical diagnosis (a patient having multiple

diseases), or content recommendation systems. The primary challenges of multi-label classification include managing label dependencies, handling potential correlations between labels, developing algorithms that can predict multiple labels concurrently, and designing appropriate evaluation metrics that capture the performance of these more intricate prediction tasks. This approach requires sophisticated machine learning models capable of understanding and leveraging the complex relationships between features and multiple potential labels, making it significantly more complex than traditional single-label classification problems.

EVALUATION MEASURE

As previously highlighted, multi-label classification increases the model's computational complexity. The challenges extend beyond algorithmic design, fundamentally impacting performance evaluation. Traditional metrics like accuracy and F1 score prove inadequate for capturing the nuanced performance of multi-label classifiers.

In this direction, researchers have developed sophisticated evaluation measures tailored explicitly to multi-label classification scenarios. The Information Contrast Model (ICM) [13] emerges as a comprehensive measure capable of addressing multi-class and

multi-label classification challenges. The Information Contrast Model examines the structural relationships between different classification labels, allowing for a more extensive evaluation that considers not just individual label predictions but also the taxonomic dependencies among these labels.

The ICM was the official evaluation metric for the third subtask. The shared task organizers developed PyEvALL[1], a dedicated Python package that enables comprehensive computation of ICM and other evaluation measures for supervised classification tasks to support participants. During this session, you will learn how to install, import the functionalities and use the ICM measure implemented in PyEvALL to train and validate the models.

FINE TUNING STRATEGIES

Like the previous session, this lab session leverages the HuggingFace API, specifically Trainer and TrainingArguments, which provides a standardized, high-level interface for fine-tuning transformer models. These APIs simplify the complex model fine-tuning process by abstracting away many low-level training details. Moreover, the AutoTokenizers and AutoModelForSequenceClassification classes are used to

[1] https://pypi.org/project/PyEvALL/

preprocess the data and load the pre-trained models. The AutoTokenizers class is crucial for text preprocessing, automatically selecting and applying the appropriate tokenization strategy based on the pre-trained model's specific requirements. Similarly, AutoModelForSequenceClassification dynamically loads the correct pre-trained transformer model architecture for sequence classification tasks. This class automatically selects the appropriate model based on the specified pre-trained checkpoint. It adds a classification head to the base transformer model. Also, it prepares the model for fine-tuning downstream tasks and handles different model architectures. Regarding the fine-tuning strategies, in this session, both the discriminative method and LoRA can be used.

Performing fine-tuning of Transformers

A fine-tuned variant involves recalibrating the weights learned by the model based on training examples from the specific task to be solved. To do this, it is necessary to define:

- The Dataset class for data handling
- The function to compute the quality measures
- The training and validation

After having these three elements, the model based on the Transformer can be retrained, followed by performing the predictions on the test dataset.

To fine-tune models for addressing the EXIST2025 shared task, specifically subtask 3, it is necessary to install all required dependencies for training and validation. Additional dependencies are required to utilize the evaluation metrics proposed by the EXIST2025 organizers.

Installing the required dependencies

```
!pip install transformers --upgrade
!pip install datasets accelerate
!pip install -U PyEvALL
```

```
#Importing the required modules
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import  AutoTokenizer, AutoModelForSequenceClassification,  Trainer,
TrainingArguments,  EarlyStoppingCallback
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
import random
import os
import pandas as pd
import sys
import tempfile
import time

#Importing the required modules to use the ICM measure

from pyevall.evaluation import PyEvALLEvaluation
from pyevall.metrics.metricfactory import MetricFactory
from pyevall.reports.reports import PyEvALLReport
from pyevall.utils.utils import PyEvALLUtils

from functools import partial
```

Like previous implementations, it is needed to extend the torch.Dataset class to manage the dataset for subtask 3. The key distinction from subtasks 1 and 2 lies in the annotation structure: while subtasks 1 and 2 use single-label annotations, this subtask

involves multi-label classification where instances can be associated with multiple labels simultaneously.

In this approach, labels are represented as a binary vector, where each position corresponds to a specific label. A value of 1 indicates the presence of a label, while 0 represents its absence. To facilitate this encoding, the MultiLabelBinarizer class from scikit-learn provides an efficient method for transforming label sets into a binary matrix representation.

EXAMPLE 1: SeximsDatasetMulti extends Dataset torch class

```
class SexismDatasetMulti(Dataset):
   def   __init__(self,   texts,   labels,   ids,   tokenizer,   max_len=128,   pad="max_length",
trunc=True,rt='pt'):
       self.texts = texts.tolist()
       self.labels = labels
       self.ids = ids
       self.tokenizer = tokenizer
       self.max_len = max_len
       self.pad = pad
       self.trunc = trunc
       self.rt = rt


   def __len__(self):
       return len(self.texts)

   def __getitem__(self, idx):
       text = str(self.texts[idx])
       inputs = self.tokenizer.encode_plus(
           text,
           add_special_tokens=True,
           max_length=self.max_len,padding=self.pad, truncation=self.trunc,
           return_tensors=self.rt
       )

       return {
           'input_ids': inputs['input_ids'].flatten(),
           'attention_mask': inputs['attention_mask'].flatten(),
           'labels': torch.tensor(self.labels[idx], dtype=torch.float),
           'id': torch.tensor(self.ids[idx], dtype=torch.long)}
```

After establishing the suitable dataset class to manage the dataset for subtask 3, the next step is to define a proper function to compute

the evaluation measures used during training. The EXIST2025 shared task employs the ICM as the official measure for subtask 3, so this metric is used.

It is important to note that the ICM measure is not natively supported in popular libraries like scikit-learn or evaluate. While it is available in the PyEvALL library, direct integration into scikit-learn's evaluation framework is not straightforward. To address this, we must implement a custom wrapper function to incorporate ICM as a validation measure during training. Example 2 demonstrates how to create such a wrapper.

EXAMPLE 2: Wrapper to compute ICM measure

```
def ICMWrapper(pred, labels, multi=False, ids=None):
   test = PyEvALLEvaluation()
   metrics=[MetricFactory.ICM.value]
   params= dict()
   fillLabel=None
   if multi:
       params[PyEvALLUtils.PARAM_REPORT]="embedded"
       hierarchy={"True":['IDEOLOGICAL-INEQUALITY',  'STEREOTYPING-DOMINANCE',  'MISOGYNY-NON-
SEXUAL-VIOLENCE', 'OBJECTIFICATION', 'SEXUAL-VIOLENCE'],
       "False":[]}
       params[PyEvALLUtils.PARAM_HIERARCHY]=hierarchy
       fillLabel = lambda x: ["False"] if len(x)== 0 else x
   else:
       params[PyEvALLUtils.PARAM_REPORT]="simple"
       fillLabel = lambda x: str(x)


   truth_name, predict_name=None, None
   if ids is None:
       ids=list(range(len(labels)))

   with tempfile.NamedTemporaryFile(mode='w', delete=False, encoding='utf-8') as truth:
       truth_name=truth.name
       truth_df=pd.DataFrame({'test_case': ['EXIST2025']*len(labels),
                       'id': [str(x) for x in ids],
                       'value': [fillLabel(x) for x in labels]})
       if multi==True:
           truth_df=truth_df.astype('object')
       truth.write(truth_df.to_json(orient="records"))

   with  tempfile.NamedTemporaryFile(mode='w', delete=False) as predict:
```

```
        predict_name=predict.name
        predict_df=pd.DataFrame({'test_case': ['EXIST2025']*len(pred),
                        'id': [str(x) for x in ids],
                        'value': [fillLabel(x) for x in pred]})
        if multi==True:
            predict_df=predict_df.astype('object')
        predict.write(predict_df.to_json(orient="records"))

    report = test.evaluate(predict_name, truth_name, metrics, **params)
    os.unlink(truth_name)
    os.unlink(predict_name)

    icm = None
    if 'metrics' in report.report:
        if 'ICM' in report.report["metrics"]:
icm=float(report.report["metrics"]['ICM']["results"]["average_per_test_case"])
    return icm
```

Example 3 illustrates how to implement custom measure computation function for use with the Trainer object during training and validation processes.

EXAMPLE 3: Quality Measures used for subtask 3

```
def compute_metrics_3(pred, lencoder):
    labels = pred.label_ids
    #preds = pred.predictions.argmax(-1)
    preds = torch.sigmoid(torch.tensor(pred.predictions)).numpy()
    preds_binary = (preds >= 0.5).astype(int)
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds_binary, average=None
    )
    acc = accuracy_score(labels, preds_binary)
    icm=ICMWrapper(lencoder.inverse_transform(preds_binary), lencoder.inverse_transform(labels),
multi=True)
    # Macro averages
    precision_macro = np.mean(precision)
    recall_macro = np.mean(recall)
    f1_macro = np.mean(f1)
    metrics = {}
    metrics.update({
        'precision_macro': precision_macro,
        'recall_macro': recall_macro,
        'f1_macro': f1_macro,
        'ICM': icm
    })
    return metrics
```

Example 4 illustrates how the RoBERTa model can be fine-tuned for subtask 3. The main distinction in subtask 3 lies in its unique label encoding and model configuration. Unlike previous subtasks, a MultiLabelBinarizer encoder transforms annotations into a binary vector representation, where each position corresponds to a potential label. The classification model (AutoModelForSequenceClassification class) reflects this by setting *nlabels=5* to match the subtask's label space and crucially configuring *ptype="multi_label_classification*". This parameter fundamentally restructures the model's learning mechanism, shifting from traditional cross-entropy to a sigmoid-based activation that enables simultaneous, non-mutually exclusive label predictions. The loss function is transformed to compute binary cross-entropy for each label independently, allowing the model to assign multiple labels to a single instance with individual probabilistic scores.

EXAMPLE 4: Training, Validation model for subtask3

```
def sexism_classification_pipeline_task3(trainInfo, devInfo, testInfo=None,
model_name='roberta-base', nlabels=5, ptype="multi_label_classification", **args):
    # Model and Tokenizer
    labelEnc= MultiLabelBinarizer()
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSequenceClassification.from_pretrained(
        model_name,
        num_labels=nlabels,
        problem_type=ptype)

    # Prepare datasets
    train_dataset = SexismDatasetMulti(trainInfo[1],
labelEnc.fit_transform(trainInfo[2]),[int(x) for x in trainInfo[0]], tokenizer )
    val_dataset = SexismDatasetMulti(devInfo[1], labelEnc.transform(devInfo[2]), [int(x) for x
in devInfo[0]], tokenizer)

    # Training Arguments
    training_args = TrainingArguments(
```

```
        report_to="none", # alt: "wandb", "tensorboard" "comet_ml" "mlflow" "clearml"
        output_dir= args.get('output_dir', './results'),
        num_train_epochs= args.get('num_train_epochs', 5),
        learning_rate=args.get('learning_rate', 5e-5),
        per_device_train_batch_size=args.get('per_device_train_batch_size', 16),
        per_device_eval_batch_size=args.get('per_device_eval_batch_size', 64),
        warmup_steps=args.get('warmup_steps', 500),
        weight_decay=args.get('weight_decay',0.01),
        logging_dir=args.get('logging_dir', './logs'),
        logging_steps=args.get('logging_steps', 10),
        eval_strategy=args.get('eval_strategy','epoch'),
        save_strategy=args.get('save_strategy', "epoch"),
        save_total_limit=args.get('save_total_limit', 1),
        load_best_model_at_end=args.get('load_best_model_at_end', True),
        metric_for_best_model=args.get('metric_for_best_model',"ICM")
    )

    # Initialize Trainer
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        #compute_metrics=compute_metrics_3,
        compute_metrics = partial(compute_metrics_3, lencoder=labelEnc),

callbacks=[EarlyStoppingCallback(early_stopping_patience=args.get("early_stopping_patience",3))
]
    )

    # Fine-tune the model
    trainer.train()

    # Evaluate on validation set
    eval_results = trainer.evaluate()
    print("Validation Results:", eval_results)

    if testInfo is not None:
      # Prepare test dataset for prediction
      test_dataset = SexismDatasetMulti(testInfo[1], [[0,0,0,0,0]] * len(testInfo[1]),  [int(x)
for x in testInfo[0]],   tokenizer)

      # Predict test set labels
      predictions = trainer.predict(test_dataset)
      #predicted_labels = np.argmax(predictions.predictions, axis=1)
      predicted_probs = torch.sigmoid(torch.tensor(predictions.predictions)).numpy()
      predicted_labels = (predicted_probs >= 0.5).astype(int)

      # Create submission DataFrame
      submission_df = pd.DataFrame({
          'id': testInfo[0],
          'label': labelEnc.inverse_transform(predicted_labels),
          "test_case": ["EXIST2025"]*len(predicted_labels)

      })
      submission_df.to_csv('sexism_predictions_task3.csv', index=False)
      print("Prediction TASK3 completed. Results saved to sexism_predictions_task2.csv")
      return model, submission_df
    return model, eval_results
```

Example 5 illustrates how to read the dataset for subtask 3, perform the models' fine-tuning, and obtain the prediction on the unannotated test sets.

EXAMPLE 5: Addressing the subtask 3 in EXIST2025 shared task

```
library_path = os.path.join(base_path, "Lab2-S1")
sys.path.append(library_path)
from readerEXIST2025 import EXISTReader

# path to the dataset, adapt this path wherever you have the dataset
dataset_path = os.path.join(base_path, "../corpora/EXIST_2025_Dataset_V0.2/")

file_train = os.path.join(dataset_path, "EXIST2025_training.json")
file_dev = os.path.join(dataset_path, "EXIST2025_dev.json")

reader_train = EXISTReader(file_train)
reader_dev = EXISTReader(file_dev)

EnTrainTask3, EnDevTask3 = reader_train.get(lang="EN", subtask="3"), reader_dev.get(lang="EN",
subtask="3")
SpTrainTask3, SpDevTask3 = reader_train.get(lang="ES", subtask="3"), reader_dev.get(lang="ES",
subtask="3")



set_seed()
modelname = "roberta-base"
params = dict()
params['num_train_epochs']=10
sexism_classification_pipeline_task3(EnTrainTask3, EnDevTask3, None, modelname, 5,
"multi_label_classification", **params )
```

ACTIVITY

The activity consists of developing and fine-tuning Transformer-based models for the "*EXIST2025: sEXism Identification In Social neTworks*" shared task, particularly subtask 3. Students must address the third subtask in English and Spanish. They can use the Transformer-based models studied in the lab sessions or any other models of their preference. In the case of the Spanish, the language-specific variants such RoBERTa for Spanish, and BETO can be used. For fine-tuning the models, use the techniques studied in the lab sessions. Developing and training your models on hardware equipped with a GPU (Graphics Processing Unit) is crucial to leverage the computational power and accelerate the training process. Deliver **the source** code and prepare a **comprehensive report** explaining your approach, strategy, and findings. Include the following aspects:

- Data preprocessing steps, if any.
- Model selection and architecture details.
- Fine-tuning techniques and hyperparameter selection strategies.
- Evaluation measures and results.
- Relevant visualizations and tables to support your work.

To achieve the maximum score, two models must be trained with two different parameter configurations for the two languages, and

the two techniques (fine-tuning and LoRA), to obtain at least 8 results = 2 (model+param) x 2 (techniques) x 2 (languages) and make a **good** report.

The Google collaborative environment (Google Colab) can be used to fine-tune the Transformer models:
https://colab.research.google.com/notebooks/welcome.ipynb#scrollTo=5fCEDCU_qrC0